CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

CHANDUBHAI S PATEL INSTITUTE OF TECHNOLOGY

SMT. KUNDANBEN DINSHA PATEL DEPARTMENT OF INFORMATION TECHNOLOGY

Academic Year: 2024-25

Semester: 5th

Course code: IT343

Course name: Operating System

Practical List

(July –Dec 2024)

Practical 1

Objective:

To gain proficiency in using various Linux commands for system administration, file handling, text processing, security, and other essential operations.

Problem:

Students need to become familiar with essential Linux commands and their options to manage and manipulate files, directories, user accounts, and processes efficiently.

Outcome:

Students will be able to perform system administration tasks, manage directories and files, process text, ensure system security, and utilize Linux utilities for various operations.

Application:

Practical application of Linux commands for system administration, file handling, security, and process management in real-world scenarios.

Exercise 1.1:

Objective: Learn to create directories and manipulate files within them.

Problem: Create directories named `DEPSTAR` and `CSPIT` with files containing DEPARTMENT and STUDENTS names. Display and manipulate these contents as specified.

Outcome: Ability to manage directories and files, display specific contents, and manipulate text within files using Linux commands.

Application: Useful for organizing data and performing text processing tasks efficiently.

Steps:

1. Create two directories named `DEPSTAR` and `CSPIT` with files `IT` and `CE`.
2. List the first 5 STUDENTS names from IT.
3. Display content of `IT` and `CE` such that each DEPARTMENT name is matched with its STUDENTS name.
4. Trim the DEPARTMENT names to the first 3 letters and save them to a new file named `initials.txt`.

Exercise 1.2:

Objective: Understand how to sort and display file contents based on specific criteria.

Problem: Create a file with student marks and display the content in descending order.

Outcome: Ability to sort and display data based on specific criteria using Linux commands.

Application: Useful for organizing and analyzing data, such as sorting student marks or other numerical data.

Steps:

1. Create a file containing students' marks.
2. Display the file content in descending order based on the marks.

Exercise 1.3:

Objective: Learn to search for patterns within files using Linux commands.

Problem: Search for valid email addresses and mobile numbers from a given student data file.

Outcome: Ability to search and extract specific patterns from text files using Linux commands.

Application: Useful for data extraction and validation tasks, such as finding contact information in large datasets.

Steps:

1. Search for valid email addresses and mobile numbers from the student data file using appropriate Linux commands.

Exercise 1.4:

Objective: Practice text manipulation using Linux commands.

Problem: Add a five-star (*****) prefix to each line and remove all blank lines from a file.

Outcome: Ability to manipulate text files, adding prefixes and removing unnecessary lines using Linux commands.

Application: Useful for formatting text files and preparing data for presentation or further processing.

Steps:

1. Add a five-star (*****) prefix to each line of a file.
2. Delete all blank lines from the file using Linux commands.

Practical 2

Objective:

To understand and practice fundamental Linux commands related to file creation, linking, and text file manipulation.

Problem:

Students need to learn how to create files, manage soft and hard links, and manipulate file content using basic Linux commands.

Outcome:

Students will be able to create and manage files, understand the difference between soft and hard links, and perform basic file content operations.

Application:

Practical application of file management, linking, and text processing in Linux, which are essential skills for system administration and general file operations.

Exercise 2.1:

Objective: Learn to create a new file and create a soft link to it.

Problem: I have a `Jupiter` file with the content "Jupiter is a planet" and create a soft link to this file in the `/tmp` directory.

Outcome: Ability to create files and soft links, understanding how soft links work in Linux.

Application: Useful for creating shortcuts and managing file references across different directories.

Steps:

1. Create a new file named `jupiter` and write the content "Jupiter is a planet" to it.
2. Create a soft link to the `jupiter` file in the `/tmp` directory.

Exercise 2.2:

Objective: Understand how to create a hard link to a file.

Problem: Create a hard link of the `jupiter` file in the `/tmp` directory.

Outcome: Ability to create hard links and understand the difference between soft and hard links.

Application: Useful for creating multiple references to the same file data within the filesystem.

Steps:

1. Create a hard link of the `jupiter` file in the `/tmp` directory.

Exercise 2.3:

Objective: Learn to view and manipulate the beginning of a file's content.

Problem: View the first 10 lines of the `mesg-new` file and output this content to a new file named `mesg-h10`.

Outcome: Ability to extract and save specific parts of a file's content using Linux commands.

Application: Useful for previewing and managing large files by focusing on specific sections.

Steps:

1. View the first 10 lines of the `mesg-new` file.
2. Output these lines to a new file named `mesg-h10`.

Exercise 2.4:

Objective: Learn to view and manipulate the end of a file's content.

Problem: View the last 20 lines of the `mesg-new` file and output this content to a new file named `mesg-t20`.

Outcome: Ability to extract and save specific parts of a file's content using Linux commands.

Application: Useful for previewing and managing large files by focusing on specific sections.

Steps:

1. View the last 20 lines of the `mesg-new` file.
2. Output these lines to a new file named `mesg-t20`.

Practical 3

Objective:

To gain hands-on experience with Linux system administration tasks, including managing environment variables, user accounts, and file permissions.

Problem:

Students need to become familiar with essential system administration tasks, such as setting environment variables, managing user accounts, and altering file permissions.

Outcome:

Students will be able to manage environment variables, create and manage user accounts, change file permissions, and monitor system user activity.

Application:

Practical application of Linux system administration tasks essential for maintaining and managing Linux-based systems.

Exercise 3.1:

Objective: Understand and manipulate basic environment variables.

Problem: with the basic environment variables (`PATH`, `USER`, `HOME`, `SHELL`) create a custom environment variable.

Outcome: Ability to view and set environment variables, understanding their roles in the system.

Application: Useful for configuring user environments and managing system settings.

Steps:

1. Study and understand the `PATH`, `USER`, `HOME`, and `SHELL` environment variables.
2. Create your own environment variable.

Exercise 3.2:

Objective: Learn to create and manage user accounts.

Problem: with a user named `hulk`, change their password, log in as `hulk`, and

create a file named `Skaar` in `hulk`'s home directory.

Outcome: Ability to manage user accounts and perform user-specific operations.

Application: Useful for system administration tasks, such as managing users and their permissions.

Steps:

1. Create a user named `hulk`.
2. Change the password for the `hulk` user.
3. Log in as `hulk`.
4. Create a file named `Skaar` in `hulk`'s home directory.

Exercise 3.3:

Objective: Learn to change file ownership and verify user details.

Problem: Change the group ownership of the file `Skaar` from `hulk` to `root` and verify the `hulk` user ID using the `id` command.

Outcome: Ability to change file ownership and verify user details using Linux commands.

Application: Useful for managing file permissions and understanding user roles in the system.

Steps:

1. Change the group ownership of the `Skaar` file from `hulk` to `root`.
2. Run the `id` command to verify the user ID of the `hulk` user.

Exercise 3.4:

Objective: Learn to monitor user activity on the system.

Problem: Use the `last` command to find out the number of users logged in.

Outcome: Ability to monitor and analyze user login activity using Linux commands.

Application: Useful for tracking user activity and ensuring system security.

Steps:

1. Run the `last` command.
2. Determine the number of users who have logged in based on the output.

Practical 4

Objective:

To learn the basics of shell scripting and how to automate tasks using shell scripts.

Problem:

Students need to write shell scripts to perform specific tasks such as finding the largest integer among given arguments and processing a student data file.

Outcome:

Students will be able to write and execute shell scripts to automate tasks and process data effectively.

Application:

Practical application of shell scripting to automate repetitive tasks, process data, and enhance productivity in a Linux environment.

Exercise 4.1:

Objective: Learn to write a shell script to find the largest integer among three given integers.

Problem: Write a shell program to find the largest integer among three integers provided as arguments.

Outcome: Ability to write a shell script that processes input arguments and performs conditional logic.

Application: Useful for creating scripts that perform calculations and decision-making based on input parameters.

Exercise 4.2:

Objective: Learn to write a shell script to process a student data file and produce specific outputs.

Problem: Create a student data file with fields: Name, Id_num, department (IT, CE, CSE), and CGPA. Write a shell script that takes this student file as input from the user and provides the following output:

- Display student Name and CGPA separated by '-'.
- Display the 3rd to the last student's Name and department with their line number.
- Display the equivalent percentage of CGPA for all students.

Outcome: Ability to write shell scripts that read and process file content, perform data extraction, and calculate derived values.

Application: Useful for data processing, reporting, and performing batch operations on files.

Practical 5

Objective:

To gain experience with shell scripting by performing text file analysis and system information retrieval.

Problem:

Students need to write shell scripts to count specific elements in a text file and retrieve various system information based on user choices.

Outcome:

Students will be able to write and execute shell scripts to analyze text files and gather system information effectively.

Application:

Practical application of shell scripting to automate text file analysis and system monitoring tasks in a Linux environment.

Exercise 5.1:

Objective: Learn to write a shell script to count various elements in a text file.

Problem: Write a shell program to count the following in a given text file:

- Number of vowels.
- Number of blank spaces.
- Number of characters.
- Number of symbols.
- Number of lines.

Outcome: Ability to write a shell script that reads a text file and performs various counts using text processing commands.

Application: Useful for analyzing text files and extracting specific information.

Exercise 5.2:

Objective: Learn to write a shell script to display various system information based

on user choices.

Problem: Write a shell script that provides the following information based on user choice:

- List of all user accounts.
- Count of all logged-on users.
- Display the group information to which the current user belongs.
- Name of currently logged-on users.
- Display the statistics of virtual memory.
- Display the network statistics and analyze the status of all network packets.

Outcome: Ability to write shell scripts that retrieve and display system information based on user input.

Application: Useful for system administration tasks, such as monitoring user activity and system performance.

Practical 6

Objective:

To learn and practice advanced shell scripting and C programming for system operations, process management, inter-process communication, deadlock avoidance, and page replacement algorithms.

Problem:

Students need to write shell scripts to manage processes, compare files, and perform system operations. Additionally, they need to write C programs for inter-process communication, deadlock avoidance, and page replacement algorithms.

Outcome:

Students will gain proficiency in writing and executing advanced shell scripts and C programs for various system operations and algorithms.

Application:

Practical application of advanced shell scripting and C programming in system administration, process management, and operating system algorithms.

Exercise 6.1:

Objective: Learn to write a shell script to manage and display process statistics.

Problem: Write a shell script to:

- Show all statistics of processes.
- List processes associated with the terminal.

- Stop a process with a given process ID.

Outcome: Ability to write shell scripts that manage and display process information using system commands.

Application: Useful for monitoring and managing system processes effectively.

Exercise 6.2:

Objective: Learn to write a shell script to check file existence and properties.

Problem: Write a shell script that:

- Takes a file name from the user.
- Checks if the file exists in the current working directory.
- Displays an appropriate message.
- If the file exists, displays its inode number and checks if the file is executable.
- Makes the file executable if it is not already.

Outcome: Ability to write shell scripts that interact with the filesystem and manage file properties.

Application: Useful for automating file management tasks and ensuring file properties are set correctly.

Exercise 6.3:

Objective: Learn to write a shell script to compare and merge files.

Problem: Write a shell script that:

- Takes two file names from the user.
- Compares the two files.
- Deletes the second file if both files are the same.
- Merges the two files into a new file if they are different.

Outcome: Ability to write shell scripts that perform file comparison and merging operations.

Application: Useful for managing and organizing files by removing duplicates and combining content.

Practical 7

Objective: Learn to write a C program for inter-process communication (IPC).

Problem: Write a C program in UNIX to implement one of the following IPC

mechanisms:

- Producer-Consumer
- Reader-Writer
- Dining Philosopher

Outcome: Ability to write C programs that implement IPC mechanisms for process synchronization and communication.

Application: Useful for developing concurrent programs that require process coordination and resource sharing.

Practical 8

Objective: Learn to write a C program for deadlock avoidance using the Banker's algorithm.

Problem: Write a C program in UNIX to implement the Banker's algorithm for deadlock avoidance.

Outcome: Ability to write C programs that implement deadlock avoidance algorithms to ensure safe resource allocation.

Application: Useful for developing systems that require safe resource management and deadlock prevention.

Practical 9

Objective: Learn to implement page replacement algorithms in C.

Problem: Implement the following page replacement algorithms in C:

- First in First Out (FIFO) Algorithm
- Least Recently Used (LRU) Algorithm
- Optimal Algorithm

Outcome: Ability to write C programs that implement and compare different page replacement algorithms for memory management.

Application: Useful for understanding and implementing memory management techniques in operating systems.

Study Practical:

Objective:

To study and gain hands-on experience with Linux architecture, operating systems, Unix/Linux commands, shell scripting, and system calls.

Problem:

Students need to understand various aspects of Linux architecture, different operating systems, Unix/Linux commands, and write shell scripts and C programs to perform specific tasks.

Outcome:

Students will gain a comprehensive understanding of Linux architecture, operating systems, Unix/Linux commands, shell scripting, and system calls.

Application:

Practical application of Linux architecture, Unix/Linux commands, shell scripting, and system calls for system administration, process management, and automation tasks.

Exercise 1:

Objective: Study the architecture and different types of operating systems.

Problem: Study the following:

- LINUX Architecture
- Types of OS: Linux, Flavors of LINUX, UNIX, MAC, Windows, etc.
- Difference Between Lollipop and Marshmallow Operating System Versions

Outcome: Understanding of Linux architecture and various operating systems, and the differences between Lollipop and Marshmallow OS versions.

Application: Useful for selecting and managing different operating systems based on their architecture and features.

Exercise 2:

Objective: Study Unix/Linux architecture and learn various Unix/Linux commands with options.

Problem: Study the Unix/Linux architecture and the following Unix/Linux commands with options:

- User Access: login, logout, passwd, exit
- Help: man, help
- Directory: mkdir, rmdir, cd, pwd, ls, mv
- Editor: vi, gedit, ed, sed
- File Handling / Text Processing: cp, mv, rm, sort, cat, pg, lp, pr, file, find, more, cmp, diff, comm, head, tail, cut, grep, touch, tr, uniq
- Security and Protection: chmod, chown, chgrp, newgrp

- Information: learn, man, who, date, cal, tty, calendar, time, bc, whoami, which, hostname, history, wc
- System Administrator: su or root, date, fsck, init 2, wall, shutdown, mkfs, mount, unmount, dump, restore, tar, adduser, rmuser
- Terminal: echo, printf, clear
- Process: ps, kill, exec
- I/O Redirection (<, >, >>), Pipe (|), *, gcc

Outcome: Ability to use various Unix/Linux commands for system administration and file management tasks.

Application: Useful for efficient system administration and file management using Unix/Linux commands.

Exercise 3

Objective: Learn to write a shell script to display user and system information.

Problem: Write a script called `hello` which outputs the following:

- Your username
- The time and date
- Who is logged on
- Also output a line of asterisks (**********) after each section.

Outcome: Ability to write a shell script that displays user and system information.

Application: Useful for creating scripts that provide quick access to user and system information.

Exercise 4

Objective: Learn to write a shell script to calculate the nth Fibonacci number.

Problem: Write a shell script that calculates the nth Fibonacci number where n will be provided as input when prompted.

Outcome: Ability to write a shell script that performs mathematical calculations and processes user input.

Application: Useful for automating mathematical calculations using shell scripts.

Exercise 5

Objective: Learn to write a shell script to find the factorial of a given number.

Problem: Write a shell script that takes a number from the user and finds the

factorial of the given number.

Outcome: Ability to write a shell script that performs factorial calculations based on user input.

Application: Useful for automating factorial calculations using shell scripts.

Exercise 6

Objective: Learn to write programs using Unix system calls.

Problem: Write programs using the following system calls of the UNIX operating system:

- fork, exec, getpid, exit, wait, stat, readdir, opendir.

1. Write a program to execute `fork()` and find out the process ID using the `getpid()` system call.
2. Write a program to execute the following system calls: `fork()`, `execl()`, `getpid()`, `exit()`, `wait()` for a process.
3. Write a program to find out the status of a named file using the `stat()` system call.

Outcome: Ability to write C programs that use Unix system calls for process management and file status checking.

Application: Useful for developing applications that require process creation, execution, and file status checking using Unix system calls.

Exercise 7

Objective: Learn to write a C program for process creation using the GCC compiler.

Problem: Write a program for process creation using C.

Outcome: Ability to write and compile C programs that create processes using the GCC compiler.

Application: Useful for understanding and implementing process creation in C using the GCC compiler.